

Recurrent Neural Networks

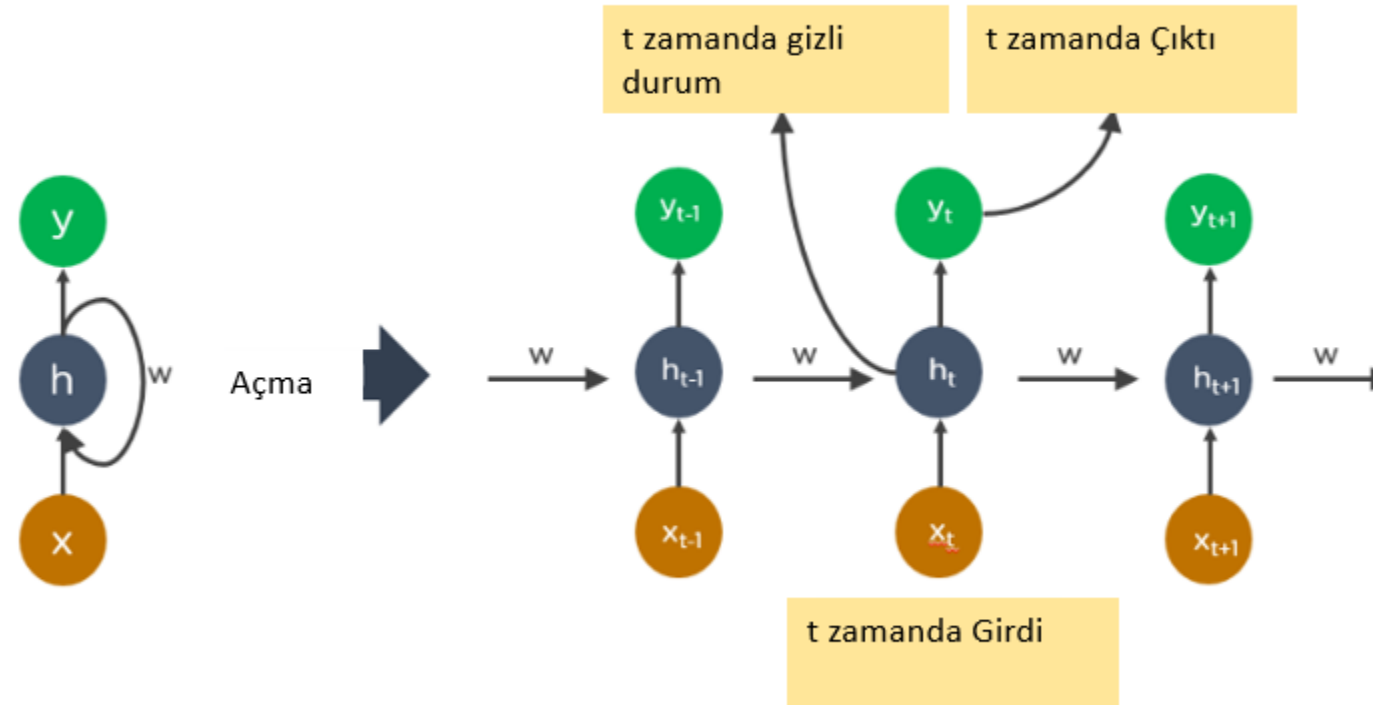
Tekrarlı Sinir Ağları

RNN's

RNN's

- Tekrarlayan Sinir Ağları (RNN), sıralı bağımlılıkları öğrenebilen ve önceki girdilerden elde edilen bilgileri dahili bellekte tutabilen bir derin öğrenme modelidir.
- Geleneksel sinir ağlarının aksine, RNN'ler zaman sırasını dikkate alır ve her girdiye aynı işlemi uygulayarak bağlamı anlamaya çalışır.
- Özellikle dil modelleme, video sınıflandırma ve konuşma tanıma gibi sıralı bilgiler içeren görevlerde başarılıdırlar.
- Ancak basit RNN'ler, uzun dizilerde bilgiyi tutmakta zorlanır. Bu sorunu aşmak için LSTM, GRU ve çift yönlü RNN gibi daha gelişmiş modeller geliştirilmiştir.

RNN's



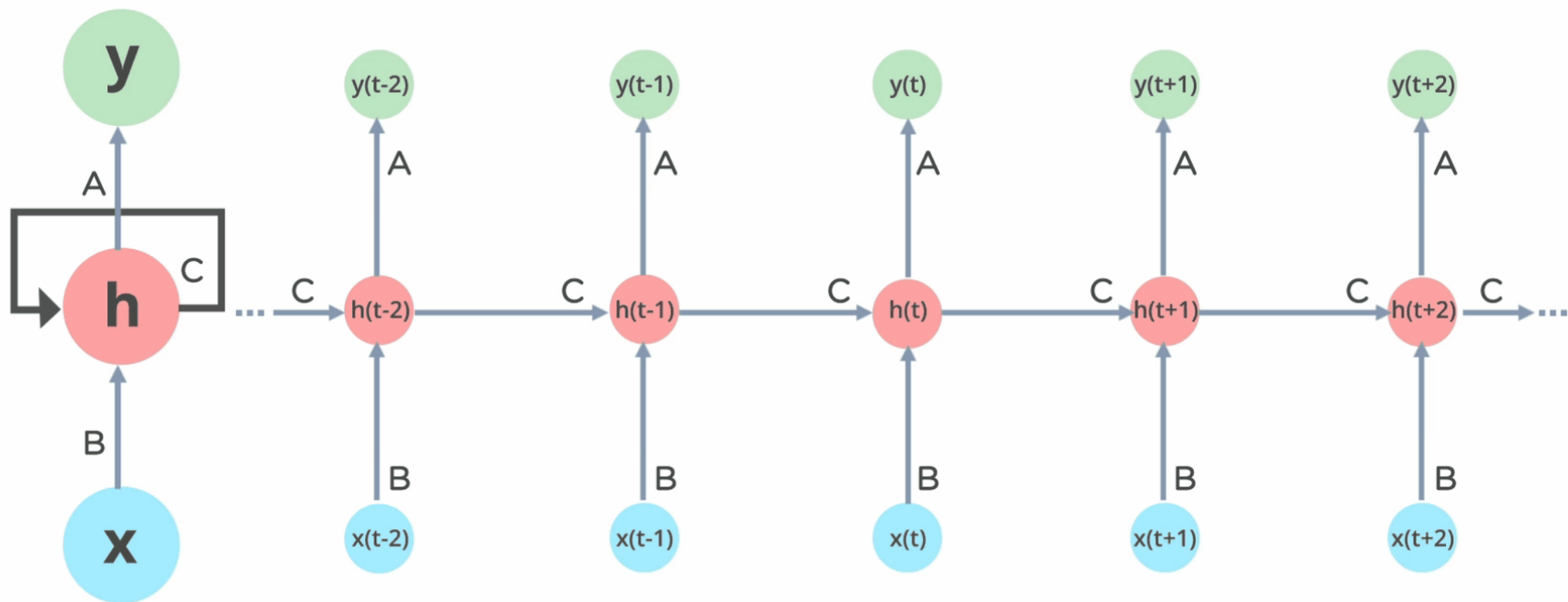
RNN'S

The diagram shows the equation $h_t = f(W \cdot h_{t-1} + U \cdot x_t + b)$ with labels pointing to its components: 'Gizli Durum' points to h_t , 'Aktivasyon Fonksiyonu(Tanh)' points to f , 'Ağırlık Matrisleri' points to W and U , 'Bias' points to b , 'Bir Önceki Gizli Durum' points to h_{t-1} , and 'Giriş Verisi' points to x_t .

$$h_t = f(W \cdot h_{t-1} + U \cdot x_t + b)$$

- Sıralı verilerle çalışabilen bir yapay sinir ağıdır. Yani;
- RNN, sıradaki bir girdiyi işlerken (örneğin bir kelime ya da zaman adımıdaki veri), daha önce işlenen girdilerin bilgisini (geçmiş bilgiyi) hatırlar ve bunu kararına dahil eder. (Bağlam Bilgisi)
- Her zaman adımında (örneğin bir kelime ya da bir anlık veri için) bir giriş alır ve bu girişle birlikte daha önce hesaplanan gizli durumu işler. (Döngüsel Yapı)
- Bu süreç tüm zaman adımları boyunca tekrar eder. Ağ, hem girdiden hem de geçmiş durumlardan bilgi öğrenir. (Yineleme)
- Her zaman adımında, bağlam bilgisine dayanarak bir çıktı üretir (örneğin bir kelimenin tahmini ya da bir sınıflandırma sonucu).

RNN's



RNN's

- Örneğin bir cümledeki kelimeleri işlerken:
- İlk kelime işlenir, ardından gizli duruma kaydedilir.
- İkinci kelime, ilk kelimenin bağlam bilgisiyle birlikte işlenir.
- Bu süreç cümledeki her kelime için devam eder.



RNN's

```
1 import numpy as np
2 from tensorflow.keras.models import Sequential
3 from tensorflow.keras.layers import SimpleRNN, Dense
4 x_train = np.random.rand(100, 10, 1)
5 y_train = np.random.rand(100, 1)
6
7 model = Sequential()
8 model.add(SimpleRNN(units=32, input_shape=(10, 1), activation='relu')) #RNN
   katman1
9 model.add(Dense(units=1, activation='linear'))
10 model.compile(optimizer='adam', loss='mse', metrics=['mae'])
11 model.summary()
12 model.fit(x_train, y_train, epochs=10, batch_size=16)
13
```

RNN'S

- Avantajları
- Sıralı verilerde bağlamı öğrenir.
- Zaman serisi verilerinde etkili modelleme yapar.
- Metin ve ses işleme gibi görevlerde performansı oldukça yüksektir.
- Geçmiş bilgileri hesaba katar ve model boyutu girdi boyutuyla artmaz.

RNN's

- Dezavantajları
- **Vanishing Gradient Problemi:** Uzun süreli bağımlılıkların öğrenilmesinde zorluk yaşanır.
- Hesaplama maliyeti yüksektir.

Long Short Term Memory Networks

Uzun Kısa Süreli Bellek Ağları

LSTM's

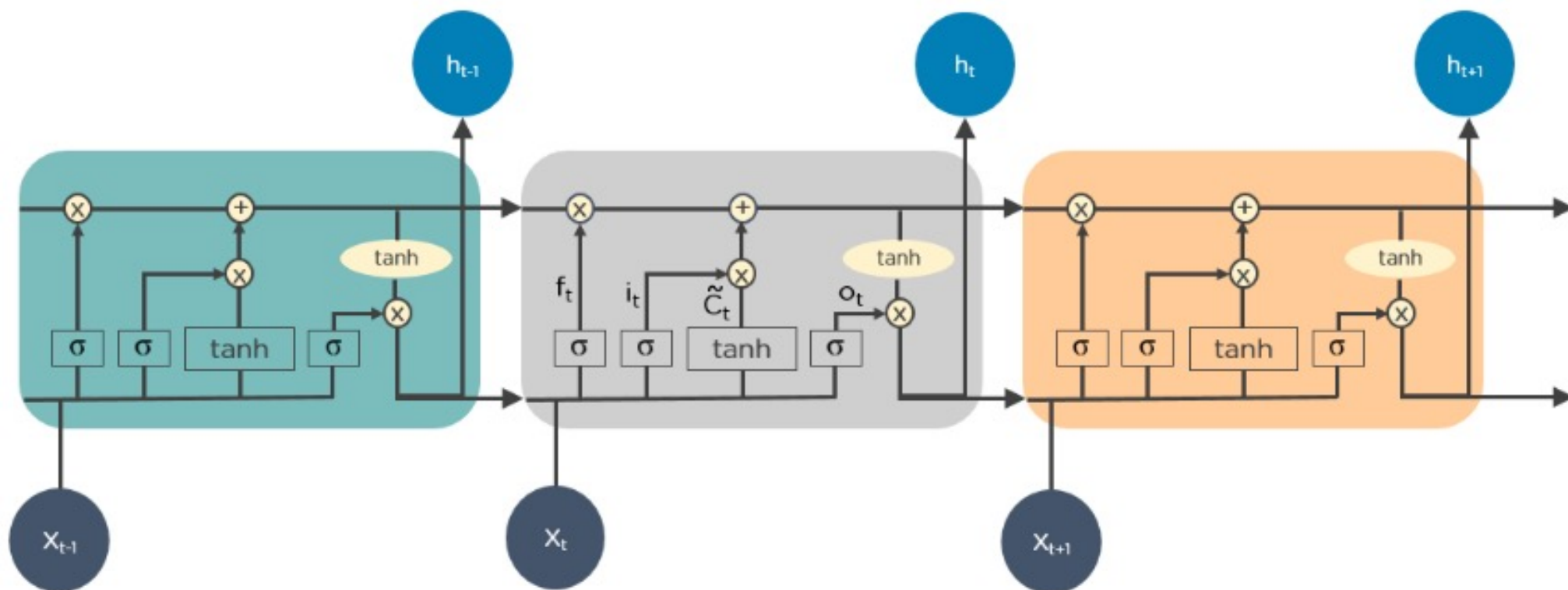
LSTM's

- LSTM'ler, uzun vadeli bağımlılıkları öğrenebilen ve ezberleyebilen bir Tekrarlayan Sinir Ağı (RNN) türüdür.
- LSTM'ler bilgileri zaman serisi içinde saklar. Önceki girdileri hatırladıkları için zaman serisi tahmininde faydalıdırlar. LSTM'ler, etkileşen dört katmanın benzersiz bir şekilde iletişim kurduğu zincir benzeri bir yapıya sahiptir. Zaman serisi tahminlerinin yanı sıra, LSTM'ler tipik olarak konuşma tanıma, müzik besteleme, dil modelleme gibi alanlarda kullanılmaktadır.

LSTM's

- Kapı mekanizmaları sayesinde, önceki bilgiyi hatırlaması veya unutması gerektiğine karar verebilir.
- **Giriş Kapısı (Input Gate):** Yeni bilgilerin hücreye girmesine karar verir.
- **Unutma Kapısı (Forget Gate):** Hangi bilgilerin unutulacağını belirler.
- **Çıkış Kapısı (Output Gate):** Hücreden hangi bilginin çıktı olarak alınacağını kontrol eder.

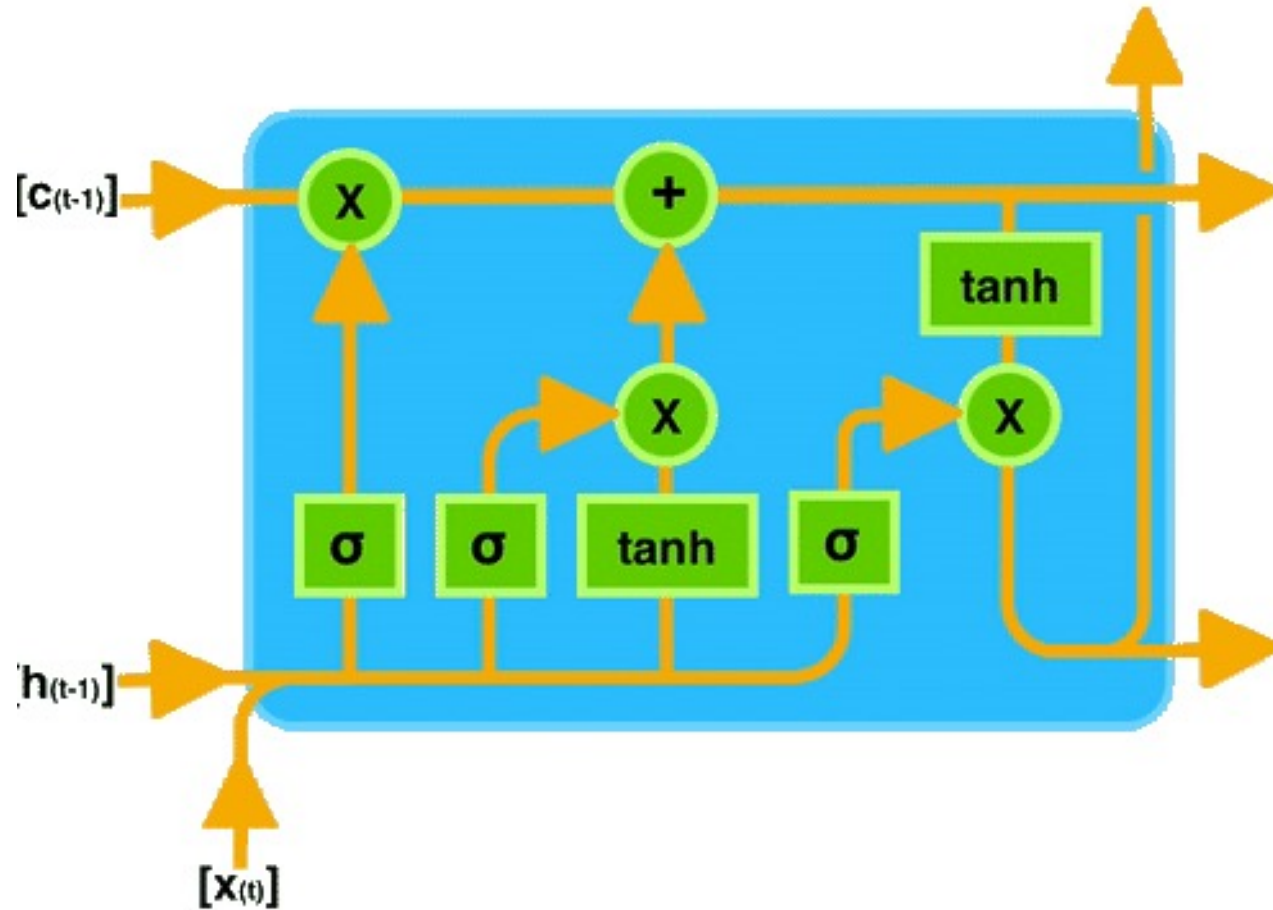
LSTM's



LSTM's

- Vanishing gradient sorununu çözer.
- Uzun dizilerde bağlam bilgilerini tutabilir.
- Standart RNN ile karşılaştırıldığında, LSTM modellerinin daha uzun dizilerdeki bilgileri tutmada ve kullanmada daha etkili olduğu kanıtlanmıştır.
- Bir LSTM ağında, belirli bir zaman adımındaki mevcut girdi ve bir önceki zaman adımındaki çıktı LSTM birimine beslenir ve bu birim daha sonra bir sonraki zaman adımına aktarılan bir çıktı üretir.

LSTM's



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

LSTM's

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
x_train = np.random.rand(100, 10, 1)
y_train = np.random.rand(100, 1)
model = Sequential()
model.add(LSTM(units=64, input_shape=(10, 1), activation='tanh',
              return_sequences=True)) # İlk LSTM katmanı
model.add(Dropout(0.2))
model.add(LSTM(units=32, activation='tanh')) # İkinci LSTM katmanı
model.add(Dropout(0.2))
model.add(Dense(units=1, activation='linear'))
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()
model.fit(x_train, y_train, epochs=20, batch_size=16)
```


Vision Transformer Method

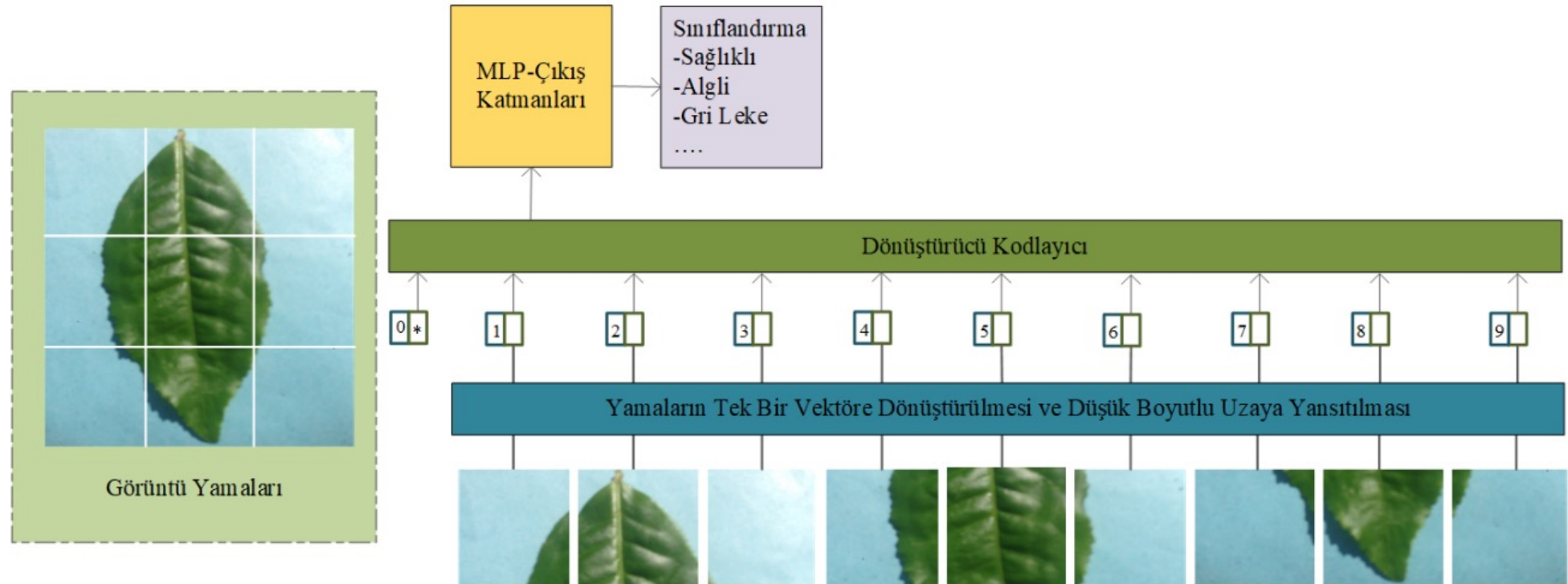
Görü Dönüştürücü Yöntemi

ViT's

ViT's

- Görü Dönüştürücü Yöntemi (ViT), veri analizi için dikkat mekanizması kullanmaktadır. Yöntemde giriş görüntüsü belirli sayıda yamaya bölünmekte ve dönüştürücülerle işlemektedir.
- **Görüntü Sınıflandırma:** Resimlerin türüne göre sınıflandırılması.
- **Görsel Nesne Tespiti:** Resimlerdeki nesnelerin konumlarının ve türlerinin belirlenmesi.
- **Segmentasyon:** Görüntüdeki her pikselin sınıflandırılması.
- **Tıbbi Görüntüleme:** Hastalıkların teşhisi için tıbbi görüntü analizi.

ViT's

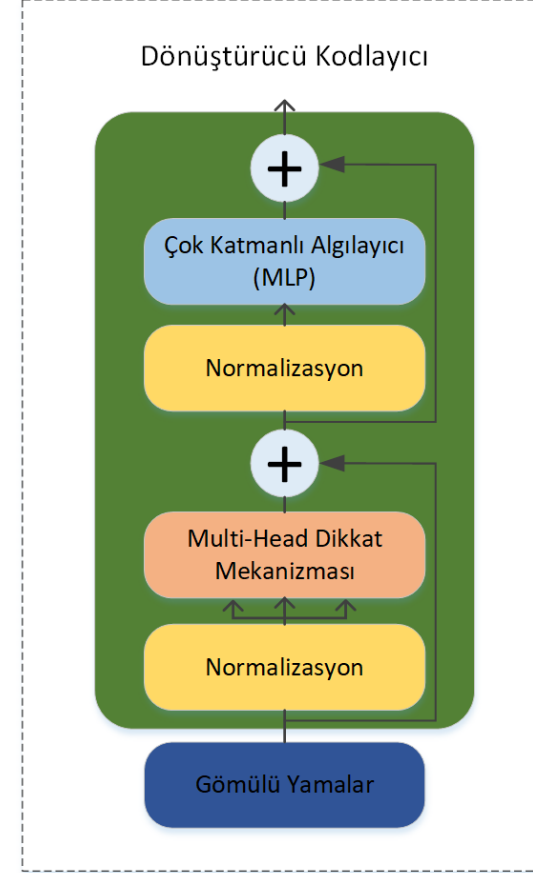


ViT's

- Görüntü verileri yamalara ayrılıp giriş katmanı aracılığıyla gömülü yamalar katmanına gönderilmektedir ve burada gömme vektörüne dönüştürülmektedir.
- 2D uzayda belirli bir konumda bulunan bu vektörlerin pozisyon bilgilerini korumak için pozisyon bilgisi atanmaktadır ve her bir işlemten sonra pozisyon bilgileri güncellenmektedir. Normalizasyon katmanı özelliklerin benzer ölçeklerde tutulmasını sağlamaktadır.
- Dikkat mekanizması, görüntü verileri içindeki ilişkileri ve önemli özellikleri modellemek için kullanılmaktadır.

Dönüştürücü Kodlayıcı

- Bu noktada dönüştürücü ağlar birçok dikkat mekanizması, normalizasyon ve beslemeli katmanlar içerebilmektedir. Her katman, özelliklerin daha yüksek seviyeli temsillerini oluşturmakta ve görüntülerin içindeki farklı özellikleri işlemeye odaklanabilmektedir.
- MLP katmanı, kodlama katmanının çıkışı olan özellik temsillerini almaktadır ve bu temsiller, özellikle dikkat mekanizması sonucunda elde edilen özelliklerden oluşmaktadır. Bu katman özellikleri daha karmaşık ve yüksek seviyede temsil etmek ve modelin verilerdeki karmaşıklığı daha iyi yakalamasını sağlamak için kullanılmaktadır.
- Son aşamada yöntem verilerin sınıflarını tahmin etmektedir.



Dikkat Mekanizması

- Dikkat mekanizması, girişteki her bir öge arasındaki ilişkileri hesaplar. Temel olarak, üç vektör üzerinde çalışır:
- **Query (Sorgu):** Anahtar bilgiyi sorgulayan vektör.
- **Key (Anahtar):** Girişin önemini belirleyen vektör.
- **Value (Değer):** Girişe karşılık gelen bilgi.

$$\text{Skor} = \text{Softmax} \left(\frac{\text{Query} \cdot \text{Key}^T}{\sqrt{d_k}} \right)$$

Dikkat Mekanizması

- **Sorgu ve Anahtar Skoru:** Her bir öğenin Query ve Key vektörleri arasında bir benzerlik skoru hesaplanır. Bu skor, öğenin önem derecesini belirler.
- **Ağırlıklı Toplama:** Bu skorlar, Value vektörleri ile çarpılarak dikkate alınması gereken bilgiler ağırlıklı olarak toplanır.

ViT's

```
import tensorflow as tf
from tensorflow.keras import layers
import numpy as np
```

Gerekli Kütüphaneler
Eklenir.

Görüntüleri küçük
parçalar haline getiren
PatchGenerator eklenir.

```
class PatchGenerator(layers.Layer):
    def __init__(self, patch_size, num_patches):
        super(PatchGenerator, self).__init__()
        self.patch_size = patch_size
        self.num_patches = num_patches

    def call(self, images):
        batch_size = tf.shape(images)[0]
        patches = tf.image.extract_patches(
            images=images,
            sizes=[1, self.patch_size, self.patch_size, 1],
            strides=[1, self.patch_size, self.patch_size, 1],
            rates=[1, 1, 1, 1],
            padding='VALID'
        )
        patches = tf.reshape(patches, [batch_size, self.num_patches, -1])
        return patches
```

patch_size: Her yamanın boyutu

num_patches: kaç tane yama olacak?

ViT's

```
class PositionalEncoding(layers.Layer):
    def __init__(self, num_patches, projection_dim):
        super(PositionalEncoding, self).__init__()
        self.positional_embeddings = layers.Embedding(
            input_dim=num_patches, output_dim=projection_dim)

    def call(self, patch_indices):
        positions = tf.range(start=0, limit=patch_indices, delta=1)
        return self.positional_embeddings(positions)
```

tf.range: Pozisyon değerlerini sıralı bir dizi olarak üretir.
self.positional_embeddings: Bu pozisyonları vektörlere dönüştürür ve her bir yamaya pozisyon bilgisini ekler.

Pozisyonel Kodlama adımı: Pozisyonel kodlama, her bir yamanın (patch) sırasını modele tanıtmak için kullanılır. Çünkü Transformer, sıralı bilgilerle çalışmak üzere tasarlanmıştır.

num_patches: Görüntünün kaç yamaya bölündüğünü ifade eder.

projection_dim: Her yama için kullanılan vektör boyutudur.

layers.Embedding: TensorFlow'un gömme katmanı, yamalar için pozisyonel vektörler oluşturur.

ViT's

```
def transformer_block(inputs, num_heads, projection_dim):
    attention_output = layers.MultiHeadAttention(num_heads=num_heads, key_dim
        =projection_dim)(inputs, inputs)
    attention_output = layers.Dropout(0.1)(attention_output)
    attention_output = layers.LayerNormalization(epsilon=1e-6)(inputs +
        attention_output)

    ff_output = layers.Dense(projection_dim * 2, activation='relu'
        )(attention_output)
    ff_output = layers.Dense(projection_dim)(ff_output)
    ff_output = layers.Dropout(0.1)(ff_output)
    ff_output = layers.LayerNormalization(epsilon=1e-6)(attention_output +
        ff_output)

    return ff_output
```

inputs: Bloğun girdi vektörüdür.

num_heads: Çok başlı dikkat (multi-head attention)

mekanizmasındaki başlık sayısını belirtir.

projection_dim: Her dikkat başlığı için projeksiyon boyutunu ifade eder.

ViT's

```
def create_vit(image_size, patch_size, num_classes, num_layers, num_heads,
               projection_dim):
    num_patches = (image_size // patch_size) ** 2
    inputs = layers.Input(shape=(image_size, image_size, 3))
    patches = PatchGenerator(patch_size, num_patches)(inputs)
    projected_patches = layers.Dense(projection_dim)(patches)
    positional_encoding = PositionalEncoding(num_patches, projection_dim)
    encoded_patches = projected_patches + positional_encoding
    for _ in range(num_layers):
        encoded_patches = transformer_block(encoded_patches, num_heads,
                                           projection_dim)
    representation = layers.LayerNormalization(epsilon=1e-6)(encoded_patches)
    representation = layers.Flatten()(representation)
    outputs = layers.Dense(num_classes, activation='softmax')(representation)
    return tf.keras.Model(inputs=inputs, outputs=outputs)
```

image_size: Giriş görüntüsünün boyutu
patch_size: Her bir yamanın (patch) boyutu
num_classes: Çıktıda sınıflandırılacak sınıf sayısı.
num_layers: Transformer bloklarının sayısı.
num_heads: Çok başlı dikkat (multi-head attention) için başlık sayısı.
projection_dim: Yamaların projeksiyon boyutu ile Transformer'ın boyutuyla eşleştirir.

PositionalEncoding: Her yamanın pozisyon bilgisini vektörlere ekler.

projected_patches + positional_encoding: Her yama artık hem içerik hem de pozisyon bilgisi taşır. ---Transformer blokları, transformer_block fonksiyonu ile tekrarlı olarak uygulanır. Bu bloklar self-attention ve feed-forward network işlemlerini içerir.-----

LayerNormalization: Modelin çıktısını normalize eder.

Flatten: Tüm yamaları tek bir düz vektöre dönüştürür.

Dense: Son tam bağlantılı katman, sınıflandırma için kullanılır.

ViT's

```
vit_model = create_vit(  
    image_size=32,  
    patch_size=4,  
    num_classes=10,  
    num_layers=8,  
    num_heads=4,  
    projection_dim=64  
)  
vit_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',  
    metrics=['accuracy'])  
vit_model.summary()
```

ViT's

- **Avantajları**
- **Bağlam Öğreniminde İyileşme:** Self-attention mekanizması sayesinde, tüm görüntü parçalarının ilişkisi aynı anda öğrenilir.
- **Mekansal Kısıtlamanın Azaltılması:** CNN'ler lokal filtrelere dayanırken, ViT tüm görüntü üzerindeki bağlamı dikkate alır.
- **Büyük Veri Setlerinde Yüksek Performans:** ViT, özellikle büyük veri kümelerinde (örneğin, ImageNet) mükemmel sonuçlar verir.

ViT's

- **Dezavantajları**
- **Büyük Veri Gereksinimi:** ViT'nin performansı, çok büyük veri setlerine bağlıdır. Küçük veri setlerinde CNN'ler daha etkili olabilir.
- **Hesaplama Maliyeti:** Self-attention mekanizması, görüntü boyutu büyüdükçe çok daha fazla hesaplama gücü gerektirir.

ViT's

- ViT ile birlikte veya ViT geliştirilerek oluşturulmuş modellerde mevcuttur.
- **DeiT (Data-efficient Image Transformers):** ViT'nin daha küçük veri setlerinde daha verimli çalışması için optimize edilmiş bir sürümü.
- **Hybrid ViT:** CNN ile Transformer bloklarını birleştiren bir yapı.

ViT's & CNN's

ViT's	CNN's
Tüm görüntü üzerindeki bağlamı öğrenir.	Lokal filtrelerle çalışır.
Büyük veri setlerinde güçlü sonuçlar üretir.	Küçük veri setlerinde daha etkilidir.
Hafıza kullanımı yüksektir.	Hafıza kullanımı daha azdır.
Hesaplama maliyeti yüksektir.	Hesaplama maliyeti ViT'e göre daha azdır.